MATH 345 Unit 4: Coding summary

Arrays, products, and orthogonality

Setup: `import numpy as np`. A vector such as `np.array([3., 4.])` has shape `(2,)`; nested row lists create a matrix.

Products: for one-dimensional vectors, `x @ q` is a dot product. For `A.shape == (m, n)` and `x.shape == (n,)`, `A @ x` has shape `(m,)`. NumPy `*` multiplies entrywise; `A.T` transposes a matrix. A one-dimensional vector's `.T` does not turn it into a column matrix.

Lengths: `np.linalg.norm(x)` computes $\|\mathbf{x}\|$. For a vector `r`, `np.linalg.norm(r)**2` and `r @ r` compute its squared norm. A zero dot product tests orthogonality, not zero length.

Projection onto a line

Normalize, project, subtract: let `f` be nonzero and let `x` have the same shape.

```
q = f / np.linalg.norm(f)
p = (x @ q) * q
v = x - p
```

Here `q` is a unit direction, `p` the projection onto its line, and `v` the perpendicular residual. The scalar `x @ q` scales `q`. Both `v @ f` and `v @ q` should be close to zero, up to roundoff.

Least squares: coefficients, fit, and residual

Input shapes: use `A` of shape `(m, n)` and `b` of shape `(m,)`.

```
xhat = np.linalg.lstsq(A, b, rcond=None)[0]
bhat = A @ xhat
r = b - bhat
```

`xhat` has shape `(n,)` and minimizes $\|A\mathbf{x} - \mathbf{b}\|$. `bhat` and `r` have shape `(m,)`: they are the fitted vector $\hat{\mathbf{b}} = A\hat{\mathbf{x}}$ and residual $\mathbf{r} = \mathbf{b} - \hat{\mathbf{b}}$. The closest reachable output is `bhat`, not the coefficient vector `xhat`.

Returned tuple: `np.linalg.lstsq` returns coefficients, residual sums of squares, rank, and singular values. `[0]` selects the coefficients, not the first coefficient. The returned residual information is not the residual vector; compute that as `b - A @ xhat`. `rcond` controls the relative cutoff for numerical rank; `rcond=None` uses NumPy's default.

Uniqueness: the fitted vector is unique. Independent columns make the coefficient vector unique; for a rank-deficient matrix, `lstsq` returns the least-squares coefficient vector of smallest Euclidean norm.

Check the residual, not just the fit

```
np.allclose(A.T @ r, np.zeros(A.shape[1]))
```

`A.T @ r` contains the n dot products with the columns of A; `np.zeros(n)` is a length-n zero vector. `np.allclose` tests agreement within tolerances. A nonzero residual can be correct; tiny orthogonality-check values such as `2.2e-16` are typical roundoff. `A @ r` is the wrong product and usually has incompatible dimensions.

Linear regression: build the design matrix

One row per data point: for inputs `t` and observed outputs `y`, each of shape `(m,)`, the model is $\hat{y} = c_0 + c_1 t$.

```
X = np.column_stack([np.ones_like(t), t])
c = np.linalg.lstsq(X, y, rcond=None)[0]
yhat = X @ c
r = y - yhat
```

Create columns: `np.ones_like(t)` creates ones with the same shape and data type as `t`. `np.column_stack` puts the arrays side by side as columns, so `X.shape` is `(m, 2)`. Its first column is the constant feature and its second is the input feature.

Read the objects: `c.shape` is `(2,)`; `c[0]` is the intercept c_0, and `c[1]` is the slope c_1. Indices start at zero. `yhat` holds the m fitted values and `r` the m vertical residuals in the data plot.

Fit diagnostics: `X.T @ r` should be close to zero; `np.linalg.norm(r)**2` is the sum of squared vertical errors. If all input values are equal, the two design columns are dependent and the intercept and slope are not uniquely determined by least squares.

Reduced QR and its shape checks

Factorization: assume `A` is $m \times n$, with $m \geq n$ and linearly independent columns.

```
Q, R = np.linalg.qr(A, mode="reduced")
np.allclose(Q.T @ Q, np.eye(Q.shape[1]))
np.allclose(Q @ R, A)
```

`Q.shape` is `(m, n)` and `R.shape` is `(n, n)`. The assignment unpacks two factors. Reduced mode keeps n columns of Q, with $Q^T Q = I_n$, and an invertible upper-triangular R. `np.eye(n)` creates I_n; `Q.shape[1]` is the column count.

Rectangular Q: when $m > n$, QQ^T is a projection matrix, not I_m. The fitted vector is `Q @ (Q.T @ b)`. Signs in NumPy's factors may differ from a hand Gram–Schmidt calculation; check orthonormality and reconstruction instead of individual signs.

Solve least squares through QR

```
x_qr = np.linalg.solve(R, Q.T @ b)
x_lstsq = np.linalg.lstsq(A, b, rcond=None)[0]
np.allclose(x_qr, x_lstsq)
```

Transformed right-hand side: `Q.T @ b` has shape `(n,)` and contains the dot products with the orthonormal columns. The solve finds $\hat{\mathbf{x}}$ from $R\hat{\mathbf{x}} = Q^T\mathbf{b}$. `np.linalg.solve` requires a square, nonsingular coefficient matrix; it solves directly without explicitly forming an inverse.

Compare methods: under the full-column-rank assumption, both coefficient vectors should agree up to floating-point error. Agreement does not imply $A\hat{\mathbf{x}} = \mathbf{b}$: the best residual can be nonzero. For regression, replace `A, b` with `X, y`. The QR route avoids forming $A^T A$ in the normal equations.